

EdgeLock

Enterprise-Grade Offline-First Face Authentication
and Liveness Detection

React Native

Expo SDK 50

TypeScript Strict

GhostFaceNet INT8

MediaPipe WASM

Technical Documentation - NHAH Hackathon 7.0

Team Binary Brains

Version 1.0 | July 2026 | github.com/DeveloperKush/NHAH-Hackathon-7.0

Contents: 1. Overview • 2. Model Architecture • 3. Liveness Detection • 4. Data Flow and Pipeline • 5. Security Architecture • 6. Performance Benchmarks • 7. Test Coverage

1. Overview

EdgeLock is an enterprise-grade offline-first face authentication system built for NHA's remote highway workforce. It runs entirely on a mobile device with no internet dependency during authentication, making it suitable for field crews in zero-connectivity zones along India's 1,40,000+ km national highway network.

The app combines on-device face recognition using a GhostFaceNet INT8 quantized model (~1 MB) with multi-factor 3D liveness detection via MediaPipe Face Mesh (468 landmarks). Every authentication event is logged with GPS coordinates, device ID, and similarity score in an AES-256-CBC encrypted local SQLite database. Logs sync automatically to AWS using a zero-loss protocol when connectivity is available.

EdgeLock addresses: (1) no cellular coverage at remote sites, (2) buddy punching with shared credentials, (3) spoofing attacks using photos or videos, and (4) absence of a tamper-proof digital audit trail.

Capability	Implementation	Source
Face Recognition	GhostFaceNet INT8, 512-dim embeddings, cosine similarity	recognition.ts
Liveness Detection	Blink (EAR), head turn (yaw), 3D depth (z std dev), random order	liveness.ts
Offline Storage	AES-256-CBC encrypted SQLite (expo-sqlite)	sqlite.ts, secureStorage.ts
Zero-Loss Sync	fetch() POST with server-confirmed purge	awsSync.ts
Face Landmarks	MediaPipe Face Mesh via WebView (468 landmarks)	mediapipeLandmarks.ts
Face Alignment	5-point similarity transform to 112x112	faceAlignment.ts

2. Model Architecture

EdgeLock uses GhostFaceNet, a lightweight neural network quantized to INT8 precision, reducing its size to ~1 MB (992 KB). Inference runs via react-native-fast-tflite using a JSI C++ binding for near-native performance. The model accepts a 112x112 RGB image (37,632 values) and outputs a 512-dimensional feature vector, L2-normalized for cosine similarity matching.

Parameter	Input Quantization	Output Dequantization
Scale	0.0078125 (1/128)	0.1412736475467682
Zero Point	-1	24
Data Type	INT8	INT8
Tensor Size	37,632 (112x112x3)	512

Quantization: $\text{int8_val} = \text{round}(\text{float_val} / \text{scale}) + \text{zero_point}$. Dequantization: $\text{float_val} = (\text{int8_val} - \text{zero_point}) * \text{scale}$. During authentication, three frames are captured 180ms apart; their embeddings are averaged and re-normalized for a more stable match.

Why GhostFaceNet?

- Size: ~1 MB INT8 quantized, far smaller than DeepFace (>100 MB) and comparable to MobileFaceNet, with better accuracy per parameter.
- Speed: Ghost Modules (linear transforms replacing expensive convolutions) plus INT8 enables real-time inference on mobile CPUs.
- Accuracy: ArcFace loss during training enforces larger angular margins between identities, producing more discriminative embeddings than softmax loss.
- Compatibility: INT8 TFLite runs on any device with TensorFlow Lite support, including mid-range Android phones.

3. Liveness Detection

EdgeLock combines four independent checks using 468 MediaPipe Face Mesh landmarks. A LivenessEngine state machine manages the challenge lifecycle with a 10-second timeout per challenge.

1. Eye Aspect Ratio (EAR)

$$\text{EAR} = (\text{dist}(p2, p6) + \text{dist}(p3, p5)) / (2 * \text{dist}(p1, p4))$$

Six landmarks per eye track openness. A blink is detected when average EAR drops below 0.33. Landmarks are sampled ~every 300ms; a single sub-threshold frame registers a blink.

2. Head Yaw Assessment

$$\text{yaw} = (\text{dLeft} - \text{dRight}) / (\text{dLeft} + \text{dRight})$$

Raw yaw is smoothed with a 5-frame rolling average. Hysteresis thresholds (ON: 0.12, OFF: 0.09) prevent flickering. Positive yaw = right turn, negative = left turn.

3. 3D Depth Consistency

Passive check on every frame: standard deviation of z-coordinates across all 468 landmarks. Flat surfaces (photos, screens) have near-zero z variance; real 3D faces exceed threshold 0.001.

4. Random Challenge Order

BLINK and HEAD_TURN are shuffled via Fisher-Yates at each session start.

State	Description	Transition
READY	Waiting for face detection + depth pass	Depth std dev > 0.001
WAITING_BLINK	User must blink	EAR < 0.33 for >= 1 frame
WAITING_HEAD_TURN	User must turn head	Smoothed yaw > 0.12 for >= 1 frame
PASSED	All challenges completed	BLINK + HEAD_TURN passed
FAILED	Timeout or spoof detected	>10s elapsed or depth check failed

4. Data Flow and Pipeline

Every authentication goes through a seven-stage pipeline, all running on-device with zero cloud dependency. Figure 1 illustrates the end-to-end flow:



Figure 1: End-to-end authentication pipeline. All stages run on-device without cloud connectivity.

Stage	Description	Source
1. Camera Capture	expo-camera captures low-res JPEG (quality 0.1, 320x240) at ~100ms intervals. Recognition uses 3 higher-quality captures (quality 0.25) 180ms apart.	frameProcessors.ts
2. MediaPipe Face Mesh	JPEG sent to WebView running MediaPipe WASM. Returns 468 landmarks (x,y,z) + confidence. XHR polyfill bypasses file:// fetch restriction.	mediapipeLandmarks.ts
3. JPEG Decode + Align	jpeg-js decodes to RGBA. 5-point similarity transform (eyes, nose, mouth) warps face to 112x112 RGB. Falls back to center crop.	faceAlignment.ts
4. Lighting Normalization	Mean brightness check: global histogram equalization per channel (normal) or CLAHE 8x8 tiles (extreme: mean <30 or >225).	imagePreProc.ts
5. GhostFaceNet Inference	112x112 RGB quantized to INT8, model.runSync() via JSI. Output dequantized to 512 float32 values, L2-normalized.	recognition.ts
6. Cosine Matching	Query embedding compared against enrolled faces using cosine similarity. Multi-threshold decision logic.	recognition.ts
7. Encrypted Log	Match result logged to AES-256-CBC encrypted SQLite with GPS, device ID, and photo thumbnail.	authLogs.ts

Threshold Logic

For single-user enrollment, a score >= 0.75 is accepted. For multi-user environments, the score must exceed 0.84, the margin over second place must be at least 0.05, and the score ratio over second place must be at least 1.08. Any score >= 0.91 is accepted regardless. Scores within 0.03 of the threshold trigger one retry capture.

5. Security Architecture

Key Lifecycle

EdgeLock implements a four-phase key lifecycle designed to protect biometric data at every stage:

- **Generate:** On first app launch, a 256-bit (32-byte) cryptographically random key is generated using `CryptoJS.lib.WordArray.random(32)`. This key is the root secret for all encryption operations.
- **Keystore:** The key is immediately stored in `react-native-encrypted-storage`, which on iOS wraps Keychain Services (`kSecAttrAccessibleWhenUnlockedThisDeviceOnly`) and on Android uses `EncryptedSharedPreferences` backed by the Android Keystore (hardware-backed TEE on devices that support it).
- **Memory Cache:** After first retrieval, the key is cached in a module-level variable for the duration of the app session. This avoids repeated secure storage reads which are orders of magnitude slower.
- **Session Destroy:** The cached key is cleared when the app process is terminated. There is no persistent plaintext copy outside the hardware-backed keystore. The key is never logged, transmitted over the network, or written to unencrypted storage.

AES-256-CBC Encryption

All biometric face embeddings are encrypted using AES-256 in Cipher Block Chaining (CBC) mode. Each encryption operation uses a random initialization vector (IV) generated by CryptoJS, ensuring that identical embeddings produce different ciphertexts. The embedding `Float32Array` (512 values = 2048 bytes) is converted to base64, encrypted as a string, and stored as ciphertext in the `enrolled_faces` SQLite table. On authentication, the ciphertext is read, decrypted, and converted back to an embedding for cosine comparison.

CBC mode was chosen for its proven track record in data-at-rest encryption and first-class support in the CryptoJS library used by the project. Unlike GCM (Galois/Counter Mode), CBC does not provide built-in authentication, but integrity is provided at the application layer: the decrypt-then-use pattern means any corruption results in an unusable embedding and a failed authentication. GCM mode is a viable future upgrade if both server-side and client-side libraries align to support it.

Threat Model

The following diagram illustrates the defense chain against the most likely threat: physical device compromise.



Figure 2: Stolen phone threat model. Data is protected at every layer.

Threat	Defense	Why It Works
Physical theft of device	Encrypted SQLite with AES-256-CBC	Ciphertext is unreadable without the key
Key extraction from storage	<code>react-native-encrypted-storage</code> (Keychain/Keystore)	Hardware-backed on TEE-capable devices
Memory dump attack	Key held in JS heap; process isolation	App sandbox prevents cross-process access
Brute force on ciphertext	256-bit key (2^{256} combinations)	Computationally infeasible
Replay of old sync data	server-confirmed purge + partial-batch safety	Only confirmed logs are ever purged

Zero-Loss Sync Protocol

When connectivity is available, unsynced auth logs are sent to an AWS endpoint. The protocol ensures no log is ever lost, even in the event of network failure or server error:

Step	Behavior
1. Network Check	<code>NetInfo.fetch()</code> confirms connectivity; returns false if offline
2. Batch Upload	Unsynced logs collected from SQLite, sent as JSON via <code>fetch()</code> POST
3. Timeout	<code>AbortController</code> with 15-second timeout; on timeout, logs are preserved
4. HTTP Validation	Logs purged only if response status is 200 (OK)
5. Server Confirmation	Purge requires <code>received_logs</code> array in response body (partial-batch safe)
6. Auto-Retry	<code>NetInfo</code> event listener triggers sync automatically on reconnection

6. Performance Benchmarks

Estimates based on codebase implementation. Exact numbers depend on device hardware, camera resolution, and lighting conditions.

Stage	Est. Duration	Notes
Camera Capture	100-150ms	captureLowResFrame() at quality 0.1
MediaPipe Landmarks	200-500ms	WebView with WASM (image decode + model)
JPEG Decode + Align	100-300ms	jpeg-js + 5-point similarity transform to 112x112
CLAHE / Histogram Eq	50-150ms	Global equalization (fast) or CLAHE (slow)
GhostFaceNet Inference	~10-50ms	model.runSync() via JSI C++ bridge
Cosine Matching	<1ms per face	O(n) with enrollment count
Total Auth (3-shot)	~2-4 sec	3 captures x 180ms apart + processing per capture
Enrollment (3 captures)	~2-3 sec	3 captures x 150ms intervals + embedding extraction

Key Metrics

Metric	Value	Detail
AI Model Size	~1 MB (992 KB)	GhostFaceNet INT8 quantized TFLite
Embedding Dimensions	512	L2-normalized unit vector
Face Landmarks	468	MediaPipe Face Mesh with x,y,z
Threshold (single user)	0.75	SIMILARITY_SINGLE_USER_THRESHOLD
Threshold (multi user)	0.84	SIMILARITY_THRESHOLD
High Confidence	0.91	SIMILARITY_HIGH_CONFIDENCE
Liveness Timeout	10s per challenge	LivenessEngine.checkTimeout()
Database Tables	2	enrolled_faces, auth_logs (expo-sqlite)
Tests Passing	163	9 suites, all passing (npm test)

7. Test Coverage

163 individual tests across 9 suites, all passing, covering every major component of the authentication pipeline. All tests run in Jest with React Native Testing Library and simulated MediaPipe landmarks.

Test Suite	Tests	Coverage
database.test.ts	11	SQLite CRUD for enrolled_faces and auth_logs
liveness.test.ts	29	EAR, yaw, depth consistency, state machine, blink/head turn
networkSync.test.ts	19	Zero-loss sync, network detection, purge safety, auto-retry
preprocessing.test.ts	30	CLAHE, histogram equalization, JPEG decode, quality gates
recognition.test.ts	23	Embedding extraction, cosine similarity, match logic, thresholds
authOrchestration.test.tsx	8	Full pipeline: liveness -> capture -> match -> log insert
e2e.test.tsx	13	End-to-end: enroll -> blink -> head turn -> auth success
enrollment.test.tsx	10	Multi-capture enrollment, embedding storage, encryption
faceAuthenticator.test.tsx	20	Component render, status updates, haptic feedback

- Liveness edge cases: state machine handles rapid blinks, partial head turns, flat-landmark spoofs. Hysteresis (ON 0.12, OFF 0.09) tested for flicker prevention.
- Zero-loss sync: logs never purged on network errors, non-200 responses, or malformed responses. Partial-batch safety verified.

- Match decision: both single-user and multi-user threshold paths tested, including margin checks, ratio checks, high-confidence fast path, and borderline retry band.
- Encryption round-trip: AES-256-CBC encrypt-then-decrypt flow tested for data integrity and correct key management.
- Build quality: TypeScript strict mode, zero compiler errors. EAS cloud builds produce Android APK and iOS IPA from any platform.